SYSTEM ANALYSIS & DESIGN

SmartListing Copilot

API Specification and Request/Response Schemas

Multimodal Listing Quality Control for Fashion E-Commerce

Responsible member	Member 4, API
Document status	Proposed design for team review. Not implemented, run or tested.
Version and date	1.1, 30 October 2026
Primary bases	Member 5 UI/UX Design Package v1.2 (28 Oct 2026); Member 2 Database Design Specification v1.1 (28 Oct 2026)
Requirements basis	FR-001 to FR-025 as quoted in the Database Design Specification (Section 15) and the UI/UX package (Sections 10 and 12). The FRS file itself was not supplied.
Architecture basis	Member 3 System Architecture (27 Oct 2026), known only through the two documents above
Deliverables	API Specification (Sections 3 to 5, 8, 9); Request/Response Schemas (Sections 6, 7, 10, Appendix A)

Design boundary. This document defines a proposed HTTP interface for the FastAPI backend. No endpoint has been implemented or executed. All JSON examples are illustrative; confidence values, identifiers, version strings and category codes are placeholders until the CV, NLP and EDA outputs exist. Values the source documents defer are marked **Proposed** or **Open** and are collected in Section 13.

Version 1.1 revision note. Clarifies UI/API integration behaviour, resolves the Review Queue latest-usable-run rule, specifies the empty-category UI state, and adds implementation examples/checks. This remains a proposed design; it has not been implemented or tested.

Contents

1 Purpose and scope
2 Source basis and assumptions
3 API conventions
4 Endpoint catalogue
5 Endpoint specifications (E-01 to E-10)
6 Data schemas
7 Enumerations
8 Validation rules
9 Error handling
10 Worked examples
11 Coordination with Database, Architecture and UI/UX
12 Requirements traceability
13 Open decisions and assumptions
14 API acceptance test checklist
Appendix A Pydantic model sketch

1. Purpose and scope

This specification defines the HTTP API that the Streamlit interface calls to submit one fashion listing, run the multimodal consistency analysis, read the result, and record the seller's decision on a suggested correction. It is the contract between the user interface (Member 5), the FastAPI backend and its CV, NLP, mismatch, scoring and suggestion modules (Member 3), and the relational database (Member 2).

In scope

- Submitting one image, a title, a description and a seller-stated category for analysis (FR-001 to FR-003).
- Returning image-supported product type and color, extracted text claims, findings with reason codes and severity, scoring, queue state and a template-based suggestion (FR-004 to FR-022).
- Recording an explicit Accept, Edit or Ignore decision with no publishing (FR-023 to FR-025).
- Re-analysis of an existing listing, retrieval of past results, the Review Queue (UC-13), supported categories, image retrieval and a health check.
- Validation rules and a uniform error contract, including the distinction between a service failure and a mismatch finding.

Out of scope

- User accounts, authentication, roles and sessions. The MVP has none (Database Design Specification, Sections 01 and 14).
- Any publish, export or marketplace-integration endpoint. None exists by design; Accept, Edit and Ignore publish nothing.
- Attributes other than product type and color. Size, material and price are not accepted, inferred or returned.
- Endpoints that delete or modify listings, runs or decisions. Retention and deletion policy is not approved (Database item D-05).
- Model training, evaluation and dataset-management endpoints.

2. Source basis and assumptions

Source	Used for	Status
Member 5 UI/UX Design Package v1.2	Screens and states, input validation (Section 12.2), findings and severity (12.3), queue rules (12.5), integration contract (Section 13), error contract (13.4)	Proposed design; stakeholder sign-off pending
Member 2 Database Design Specification v1.1	Eight tables, column names and types, enumerations (Appendix A), constraints, lifecycle and deletion behaviour, FR coverage (Section 15)	Proposed; PostgreSQL provisional; not executed against PostgreSQL
Functional Requirements Specification (FR-001 to FR-025)	Requirement statements, quoted in Member 2 Section 15	Final revised; approval pending. Used only as quoted.
Member 3 System Architecture (27 Oct 2026)	Streamlit to FastAPI to CV/NLP in parallel, then mismatch, scoring and suggestion; only FastAPI reads and writes the database	Not supplied directly; taken from Member 2 Sections 03 and 16
Member 1 UML, Member 6 test plan, NFR document	Not supplied	Cross-checks against these remain to be done

Working assumptions. (1) FastAPI with Pydantic validates requests and responses, as stated in Member 2 Section 12. (2) The analysis pipeline runs synchronously inside a single request in the MVP (see open decision A-01). (3) Every identifier is the database identifier (BIGINT, serialised as a JSON integer). (4) Where the sources

defer a value, this document proposes a configurable constant and marks it **Proposed**; it does not treat the value as decided.

3. API conventions

Base path	`/api/v1`. The version is part of the path. Breaking changes require a new version.
Transport and format	HTTP/1.1 with JSON (`application/json`, UTF-8) for all responses. Requests are JSON, except E-03, which is `multipart/form-data` because it carries an image.
Authentication	None. The MVP is local and has no user accounts. Network exposure must be limited to the local deployment; no endpoint accepts credentials.
Identifiers	`listing_id`, `analysis_run_id`, `suggestion_id`, `decision_id`, `finding_id`, `claim_id` and `prediction_id` are integers taken from the database.
Timestamps	ISO 8601 in UTC with a trailing `Z`, for example `2026-10-30T09:14:02Z`. Source columns are TIMESTAMPTZ.
Field naming	`snake_case` for all JSON fields and enumeration values, except finding severity (`Critical`, `Major`, `Minor`) and reason codes (`UPPER_SNAKE_CASE`), which follow the approved vocabularies.
Null handling	Fields that can be absent are always present with `null`. A missing key never means null. `null` never means a negative finding.
Request tracing	The client may send `X-Request-ID`. The server echoes it, or generates one, in the response header and in every error body.
Processing model	Synchronous. E-03 and E-05 return after the pipeline finishes. Stage-by-stage progress is not exposed (open decision A-01).
Idempotency	GET endpoints are safe and idempotent. E-03 creates a new listing on every successful call; the interface disables the submit control while a request is pending. E-05 creates a new run on every call. E-08 records at most one decision per suggestion.
Safety	No endpoint publishes, edits or deletes a listing. Image file paths are never accepted from or returned to the client.

4. Endpoint catalogue

ID	Method	Path	Purpose	Requirements
E-01	GET	`/api/v1/health`	Liveness and readiness check	NFR reliability
E-02	GET	`/api/v1/categories`	List supported product categories for the seller-category control	FR-003, FR-004, FR-015
E-03	POST	`/api/v1/listings/analyze`	Submit one listing and run the analysis	FR-001 to FR-022
E-04	GET	`/api/v1/listings/{listing_id}`	Read a stored listing snapshot	FR-001 to FR-003, FR-022
E-05	POST	`/api/v1/listings/{listing_id}/analyses`	Re-analyse a stored listing (new run, no overwrite)	FR-004 to FR-022
E-06	GET	`/api/v1/listings/{listing_id}/analyses`	List the analysis runs of a listing	FR-018 to FR-020
E-07	GET	`/api/v1/analyses/{analysis_run_id}`	Read the full result of one analysis run	FR-004 to FR-022

ID	Method	Path	Purpose	Requirements
E-08	POST	`/api/v1/suggestions/{suggestion_id}/decision`	Record Accept, Edit or Ignore	FR-023 to FR-025
E-09	GET	`/api/v1/review-queue`	List the latest result per listing, filtered by queue state	FR-020, UC-13
E-10	GET	`/api/v1/listings/{listing_id}/image`	Return the stored image for display	FR-001, FR-022

Writes happen only in E-03, E-05 and E-08. All other endpoints are read-only.

5. Endpoint specifications

5.1 E-01 Health check

Method and path	`GET /api/v1/health`
Purpose	Report whether the API process, the database and the model and lexicon store are reachable. Used by the interface at start-up and by deployment tooling.
Requirements	NFR reliability (identifiers to be confirmed against the NFR document)
Request format	No body
Database effect	None. The database is probed with a read-only query.

Responses

Status	Condition	Body
200	All components reachable	`{"status": "ok", "version": "1.0.0", "components": {"database": "ok", "model_store": "ok"}}`
503	Any component unreachable	Same shape with `"status": "unavailable"` and the failing component set to `"unavailable"`. This is the one response that does not use the error envelope.

Behaviour and rules

- The endpoint takes no parameters and never reads or writes listing data.
- A 503 here means the service cannot analyse; it says nothing about any listing.

5.2 E-02 List supported categories

Method and path	`GET /api/v1/categories`
Purpose	Return the controlled product-type list so the interface can populate the seller-stated category control from one shared source (UI/UX Section 12.1).
Requirements	FR-003, FR-004, FR-015
Request format	No body
Database effect	Reads `supported_category`.

Parameters

Name	In	Type	Required	Rules and description
include_inactive	query	boolean	No	Default `false`. When `true`, rows with `is_active = false` are also returned.

Responses

Status	Condition	Body
200	Success	`{"items": [CategoryOut], "count": integer}`. See Section 6.13.
422	Invalid query value	Error envelope, Section 9.

Behaviour and rules

- `supported_category` is intentionally unseeded until EDA confirms exactly five classes (Database item D-02). Until then the endpoint returns `{"items": [], "count": 0}`. The interface must decide whether to fall back to its provisional list or to block submission (open decision A-05).
- After approval, exactly five rows are active.

5.3 E-03 Submit a listing and run the analysis

Method and path	`POST /api/v1/listings/analyze`
Purpose	Validate and store one listing snapshot, run the CV and NLP branches, compare the results, score and route the listing, generate a template-based suggestion when safe, persist the evidence, and return the full result.
Requirements	FR-001 to FR-022. UI/UX screens 6.1 to 6.8.
Request format	`multipart/form-data`
Database effect	Inserts `product_listing` and `analysis_run`; then `visual_prediction`, `text_claim`, `analysis_finding` and, when generated, `suggested_correction`; finally updates `analysis_run` (status, queue state, level, score, image quality, versions, completion time).

Request body

Part	Type	Required	Rules
image	file	Yes	Exactly one file part. Allowed formats and size limit are configurable (Section 8.2). Must decode as an image.
title	string	Yes	Not blank after trimming. Stored exactly as submitted. Maximum length is a configurable constant.
description	string	Yes	Not blank after trimming. Seller-entered or synthetic text, not a dataset field. Stored as submitted.
seller_category	string	Yes	Not blank, at most 120 characters. Kept raw even when unsupported; compared with the category set by the analysis, not rejected.
text_origin	string	No	One of `seller_input`, `manual_demo`, `synthetic_evaluation`. Default `seller_input`.
image_origin	string	No	**Proposed.** Allowed values are not enumerated in the database specification (open decision A-04). Default set by the server.
dataset_product_id	string	No	At most 80 characters. For evaluation listings that come from the dataset.

Responses

Status	Condition	Body
201	Pipeline completed or partially completed	`AnalysisResult` (Section 6.10) with `status` `completed` or `partial`. Header `Location: /api/v1/analyses/{analysis_run_id}`.
400	Body is not valid multipart	`MALFORMED_REQUEST`
413	Body exceeds the hard request limit	`REQUEST_TOO_LARGE`. Raised before the body is fully read.
422	Any field or image rule fails	`VALIDATION_ERROR` with one `details` entry per problem. Nothing is stored.
503	No module produced a usable result, or storage failed	`ANALYSIS_FAILED` or `SERVICE_UNAVAILABLE`. Listing and failed run are kept when they were already created, so E-05 can retry.
504	Pipeline exceeded its time limit	`ANALYSIS_TIMEOUT`; the run is marked `failed`.
500	Unexpected error	`INTERNAL_ERROR`

Behaviour and rules

- **Processing order.** (1) Parse the multipart body. (2) Validate all text fields and the image, collecting every error. (3) Store the image file and compute its SHA-256. (4) Insert `product_listing`. (5) Insert `analysis_run` with status `running`. (6) Run the CV and NLP modules in parallel in memory. (7) Run mismatch detection, scoring and triage, and suggestion generation in memory. (8) Write all evidence rows and update the run in one transaction. (9) Build and return the response.
- **Nothing is stored when validation fails.** If the database insert fails after the image file was written, the file is removed.
- **Partial results.** If one of CV or NLP fails and the other succeeds, the run is saved with status `partial`, the queue state is `needs_review`, no suggestion is generated, and the response is still 201 (Database Section 13).
- **Complete failure.** If neither branch produces a result, the run is saved with status `failed`, the listing is kept, and the response is a 503 error that includes `listing_id`, `analysis_run_id` and the retry route in `context`.
- **Blur, resolution and brightness are findings, not errors.** A blurred or dark image is analysed and reported with `IMAGE_BLUR` or an image-quality note; only an unreadable file is rejected (FR-006).
- **Unsupported seller category is a finding, not an error.** It yields `UNSUPPORTED_CATEGORY` and routes the listing to Needs Review (FR-015).

5.4 E-04 Get a listing

Method and path	`GET /api/v1/listings/{listing_id}`
Purpose	Return the immutable listing snapshot (original text, seller category, provenance and an image URL) for the Submitted Listing card and the before/after comparison.
Requirements	FR-001 to FR-003, FR-022
Request format	No body
Database effect	Reads `product_listing` and the newest `analysis_run` id for the listing.

Parameters

Name	In	Type	Required	Rules and description
listing_id	path	integer	Yes	Positive integer.

Responses

Status	Condition	Body
200	Found	`ListingOut` (Section 6.1) plus `latest_analysis_run_id` (integer or null).
404	No such listing	`LISTING_NOT_FOUND`
422	`listing_id` is not a positive integer	`VALIDATION_ERROR`

5.5 E-05 Re-analyse a listing

Method and path	`POST /api/v1/listings/{listing_id}/analyses`
Purpose	Create a new analysis run for an existing listing. Used to retry after a service failure without re-uploading the image, or to re-run after a model or ruleset update. Earlier runs are never overwritten.
Requirements	FR-004 to FR-022; Database Section 13 (re-analysis creates a new run)
Request format	Empty body (an empty JSON object is accepted)
Database effect	Inserts a new `analysis_run` and its child rows. `product_listing` is not modified.

Parameters

Name	In	Type	Required	Rules and description
listing_id	path	integer	Yes	Positive integer.

Responses

Status	Condition	Body
201	Run completed or partial	`AnalysisResult` (Section 6.10). `Location` header points to E-07.
404	No such listing	`LISTING_NOT_FOUND`
404	Image file missing from storage	`IMAGE_NOT_FOUND`
503 / 504 / 500	As in E-03	Same error codes and `context`.

Behaviour and rules

- Editing the title, description or category is **not** re-analysis. Listings are immutable snapshots, so a corrected listing is submitted again through E-03 and becomes a new listing.
- A decision recorded against a suggestion from an earlier run stays attached to that run.

5.6 E-06 List analysis runs of a listing

Method and path	`GET /api/v1/listings/{listing_id}/analyses`

Purpose	Return the history of runs for one listing, newest first.
Requirements	FR-018 to FR-020
Request format	No body
Database effect	Reads `analysis_run`, with counts from `analysis_finding` and existence of `suggested_correction`.

Parameters

Name	In	Type	Required	Rules and description
listing_id	path	integer	Yes	Positive integer.
limit	query	integer	No	1 to 100. Default 20.
offset	query	integer	No	0 or more. Default 0.

Responses

Status	Condition	Body
200	Success (may be an empty list)	`{"items": [AnalysisSummaryItem], "total": integer, "limit": integer, "offset": integer}`. See Section 6.11.
404	No such listing	`LISTING_NOT_FOUND`
422	Invalid query value	`VALIDATION_ERROR`

5.7 E-07 Get an analysis result

Method and path	`GET /api/v1/analyses/{analysis_run_id}`
Purpose	Return the complete stored result of one run in the same shape as the POST responses.
Requirements	FR-004 to FR-022
Request format	No body
Database effect	Reads `analysis_run`, `product_listing`, `visual_prediction`, `text_claim`, `analysis_finding`, `suggested_correction`, `seller_decision`. Comparison rows and summary counts are derived at read time.

Parameters

Name	In	Type	Required	Rules and description
analysis_run_id	path	integer	Yes	Positive integer.

Responses

Status	Condition	Body
200	Found	`AnalysisResult` (Section 6.10).
404	No such run	`ANALYSIS_NOT_FOUND`
422	Not a positive integer	`VALIDATION_ERROR`

Behaviour and rules

- A run with status `running` returns 200 with empty evidence arrays and null `scoring` fields. A run with status `failed` returns 200 with `status: "failed"`; the 503 error applies only to the request that triggered the failure.
- A GET never recomputes anything. It returns what was stored at completion.

5.8 E-08 Record a decision on a suggestion

Method and path	`POST /api/v1/suggestions/{suggestion_id}/decision`
Purpose	Record the seller's explicit choice to accept, edit or ignore a suggested correction. Nothing is published.
Requirements	FR-023, FR-024, FR-025
Request format	`application/json`
Database effect	Inserts one `seller_decision` row. The `product_listing` row and the `suggested_correction` row are not modified.

Parameters

Name	In	Type	Required	Rules and description
suggestion_id	path	integer	Yes	Positive integer.

Request body

Field	Type	Required	Description / database source
decision	string	Yes	`accept`, `edit` or `ignore`.
final_title	string|null	Edit only	Required and not blank when `decision` is `edit`. Must be absent or null otherwise.
final_description	string|null	Edit only	Required and not blank when `decision` is `edit`. Must be absent or null otherwise.

Responses

Status	Condition	Body
201	Decision recorded	`DecisionOut` (Section 6.9). `published` is always `false`.
404	No such suggestion	`SUGGESTION_NOT_FOUND`
409	A decision already exists for this suggestion	`DECISION_ALREADY_RECORDED`, with the existing `decision_id` in `context`.
422	Payload breaks the rules above	`VALIDATION_ERROR` with issue `DECISION_FIELDS_REQUIRED` or `DECISION_FIELDS_NOT_ALLOWED`.

Behaviour and rules

- The rules mirror the database check on `seller_decision`: Accept and Ignore store no final text; Edit stores both final fields.
- The response contains a derived `resulting_text`: the suggested wording for Accept, the seller's wording for Edit, the original wording for Ignore. It is information for the interface only; it is not written anywhere.

- One decision per suggestion is enforced by a unique constraint. Changing a decision is not supported in the MVP (edit-history policy, Database item D-05).
- There is no endpoint that applies the text to a marketplace listing. The interface must state that nothing is published.

5.9 E-09 Review queue

Method and path	`GET /api/v1/review-queue`
Purpose	Return the latest completed or partial analysis for each listing, optionally filtered by queue state. The queue is a derived read model; no queue table exists (Database Figure 8).
Requirements	FR-020, UC-13
Request format	No body
Database effect	Reads `analysis_run` (latest completed or partial run per listing, selected with a window function), `product_listing`, `analysis_finding`, `suggested_correction`, `seller_decision`.

Parameters

Name	In	Type	Required	Rules and description
queue_state	query	string	No	`ready_to_publish`, `needs_review` or `critical_mismatch`. Omit for all three.
limit	query	integer	No	1 to 100. Default 20.
offset	query	integer	No	0 or more. Default 0.

Responses

Status	Condition	Body
200	Success (may be empty)	`{"items": [ReviewQueueItem], "total": integer, "limit": integer, "offset": integer}`. See Section 6.12.
422	Invalid query value	`VALIDATION_ERROR`

Behaviour and rules

- Items are ordered by run creation time, newest first. A severity-based ordering is not defined by the sources.
- The queue selects the latest completed or partial run per listing, then applies the queue_state filter. A newer failed or running retry does not hide the most recent usable result. This matches the Database Design Specification's latest relevant completed/partial run rule and the UI/UX queue workflow.
- `ready_to_publish` is a review label only. It does not publish anything.

5.10 E-10 Get the listing image

Method and path	`GET /api/v1/listings/{listing_id}/image`
Purpose	Return the stored product image so the interface can show the submitted listing without direct file access.
Requirements	FR-001, FR-022
Request format	No body

Database effect	Reads `product_listing.image_ref`; the file is read from managed storage.

Parameters

Name	In	Type	Required	Rules and description
listing_id	path	integer	Yes	Positive integer.

Responses

Status	Condition	Body
200	Found	Binary image with the stored `Content-Type` and `X-Content-Type-Options: nosniff`.
404	Unknown listing	`LISTING_NOT_FOUND`
404	Listing exists but the file is missing	`IMAGE_NOT_FOUND`

Behaviour and rules

- The client supplies only `listing_id`. The server resolves the path from `image_ref`, so path traversal through this endpoint is not possible.
- `image_ref` is never returned in JSON; clients use the relative `image_url` field.

6. Data schemas

Types use JSON terms. `string|null` means the key is always present and may be null. The last column names the source column or states that the value is derived. Schema names correspond to the Pydantic models in Appendix A and to the typed objects named in the architecture (Section 11.3).

6.1 ListingOut

Field	Type	Required	Description / database source
listing_id	integer	Yes	`product_listing.listing_id`
title	string	Yes	Submitted title, unchanged
description	string	Yes	Submitted description, unchanged
seller_category	string	Yes	Raw seller-supplied category, including unsupported values
text_origin	string	Yes	`seller_input`, `manual_demo` or `synthetic_evaluation`
image_origin	string	Yes	`product_listing.image_origin`; values pending (A-04)
dataset_product_id	string|null	Yes	Dataset identifier when available
image_url	string	Yes	Relative URL of E-10. Replaces `image_ref`, which is never exposed
image_sha256	string|null	Yes	64-character hex checksum
created_at	string	Yes	Submission time

6.2 ImageQuality

Field	Type	Required	Description / database source
width_px	integer|null	Yes	`analysis_run.image_width_px`; null if not decoded
height_px	integer|null	Yes	`analysis_run.image_height_px`
is_blurred	boolean|null	Yes	`analysis_run.is_blurred`. Null means the check was not obtained, not that the image is sharp
brightness_status	string|null	Yes	`analysis_run.brightness_status`. **Proposed** values `normal`, `too_dark`, `too_bright` (A-08)

6.3 VisualEvidence and VisualPrediction

`VisualEvidence` holds the selected prediction for each attribute and the remaining predictions kept as evidence.

Field	Type	Required	Description / database source
product_type	VisualPrediction|null	Yes	Row with `attribute = product_type` and `is_selected = true`; null if none
color	VisualPrediction|null	Yes	Row with `attribute = color` and `is_selected = true`; null if none
other_predictions	VisualPrediction[]	Yes	Rows with `is_selected = false`: CLIP baseline or fallback, dominant-color corroboration

Field	Type	Required	Description / database source
prediction_id	integer	Yes	`visual_prediction.prediction_id`
attribute	string	Yes	`product_type` or `color`
model_source	string	Yes	**Proposed** codes `cv`, `clip`, `dominant_color` (database lists the sources but not the codes)
prediction_state	string	Yes	`predicted` or `abstained`
value	string|null	Yes	Display value: category display name for product type, normalised color label for color. Null when abstained
category_code	string|null	Yes	`predicted_category_code`; product type only
confidence	number|null	Yes	0 to 1. Confidence, not verified correctness (FR-013)
is_selected	boolean	Yes	Chosen evidence for the attribute

6.4 TextClaim

Field	Type	Required	Description / database source
claim_id	integer	Yes	`text_claim.claim_id`
attribute	string	Yes	`product_type` or `color`
source_field	string	Yes	`title` or `description`
matched_text	string	Yes	Exact phrase in the source field
canonical_value	string	Yes	Normalised value; may be outside the supported set
script_route	string	Yes	**Proposed** `en` (English vocabulary) or `ar` (fixed Arabic lexicon), FR-008

6.5 Finding

Field	Type	Required	Description / database source
finding_id	integer	Yes	`analysis_finding.finding_id`
reason_code	string	Yes	One of six approved codes (Section 7). Every finding carries exactly one (FR-016)
attribute_scope	string	Yes	**Proposed** `product_type`, `color`, `image` or `listing` (A-08)
severity	string|null	Yes	`Critical`, `Major` or `Minor` for true mismatches; null for review flags such as `LOW_CONFIDENCE`
confidence	number|null	Yes	Warning confidence when available
visual_value	string|null	Yes	Image-supported evidence used in the explanation
claimed_value	string|null	Yes	Text claim used in the explanation
explanation	string	Yes	Plain-language explanation from the template (FR-016). Never empty

6.6 Comparison

Derived at response time from the stored predictions, claims and findings. Not stored, so the interface never types counts by hand (UI/UX Section 13.3).

Field	Type	Required	Description / database source
rows	ComparisonRow[]	Yes	Exactly two rows when the run is complete: `product_type` and `color`
summary	ComparisonSummary	Yes	Counts for the four tiles

Field	Type	Required	Description / database source
attribute	string	Yes	`product_type` or `color`
image_supported	object	Yes	`{"available": boolean, "value": string|null, "confidence": number|null}`. `available` is false when the prediction abstained or fell below the threshold
text_claim	object	Yes	`{"stated": boolean, "value": string|null}`. `stated` is false when no claim was extracted
status	string	Yes	`consistent`, `possible_mismatch`, `not_stated` or `uncertain`
finding_id	integer|null	Yes	The finding that explains a `possible_mismatch` or `not_stated` status

Field	Type	Required	Description / database source
attributes_checked	integer	Yes	Number of rows (2)
consistent	integer	Yes	Rows with status `consistent`
needs_review	integer	Yes	Rows with status `possible_mismatch` or `not_stated` (**Proposed** grouping)
uncertain	integer	Yes	Rows with status `uncertain`

Row status rules.

Condition	Status
Image-supported value is unavailable (abstained, null or below the confidence threshold)	`uncertain`. No match or mismatch is inferred (FR-014).
Image value is available and no claim was extracted for the attribute	`not_stated`, with a `MISSING_ATTRIBUTE` finding when that optional feature is enabled (FR-012).
Both values present and equivalent (same type, or same color family including ignored near-shades)	`consistent`
Both values present and contradictory (any TYPE_MISMATCH or COLOR_MISMATCH finding, including Minor)	`possible_mismatch`

6.7 Scoring

Field	Type	Required	Description / database source
quality_score	number|null	Yes	0 to 100; null while the optional Listing Quality Score is deferred (FR-018)
inconsistency_level	string|null	Yes	`low`, `medium` or `high` (FR-019). Heuristic, never a return probability
queue_state	string|null	Yes	`ready_to_publish`, `needs_review` or `critical_mismatch` (FR-020). Null only when the run failed or is running

6.8 Suggestion

`suggestion` is an object when a template correction exists and null otherwise. When it is null, `suggestion_unavailable_reason` explains why; when it is an object, that field is null.

Field	Type	Required	Description / database source
suggestion_id	integer	Yes	`suggested_correction.suggestion_id`
original	object	Yes	`{"title": string, "description": string}` from the listing snapshot
suggested	object	Yes	`{"title": string, "description": string}`; template output, never applied automatically
changed_terms	object[]	Yes	Derived word-level difference for highlighting: `[{"field": "title"|"description", "from": string, "to": string}]`
change_explanation	string	Yes	`suggested_correction.change_explanation`; why the wording differs (FR-022)
decision	DecisionOut|null	Yes	The recorded decision, or null if the seller has not chosen

6.9 DecisionOut

Field	Type	Required	Description / database source
decision_id	integer	Yes	`seller_decision.decision_id`
suggestion_id	integer	Yes	Target suggestion

Field	Type	Required	Description / database source
analysis_run_id	integer	Yes	Run that produced the suggestion
listing_id	integer	Yes	Listing analysed
decision	string	Yes	`accept`, `edit` or `ignore`
final_title	string|null	Yes	Set only for `edit`
final_description	string|null	Yes	Set only for `edit`
decided_at	string	Yes	Decision time
resulting_text	object	Yes	Derived: `{"title", "description", "basis"}` where `basis` is `suggested`, `seller_edit` or `original`
published	boolean	Yes	Always `false`. No publishing exists

6.10 AnalysisResult

Returned by E-03, E-05 and E-07. It combines everything the results screen needs in one response.

Field	Type	Required	Description / database source
analysis_run_id	integer	Yes	`analysis_run.analysis_run_id`
listing_id	integer	Yes	`analysis_run.listing_id`
status	string	Yes	`running`, `completed`, `partial` or `failed`
created_at	string	Yes	Run creation time
completed_at	string|null	Yes	Null while running
listing	ListingOut	Yes	Embedded so the interface needs no second call
versions	object	Yes	`{"cv_model_version", "nlp_lexicon_version", "ruleset_version"}`, each string or null
image_quality	ImageQuality	Yes	Section 6.2
visual_evidence	VisualEvidence	Yes	Section 6.3
text_claims	TextClaim[]	Yes	Section 6.4; empty when nothing was extracted
findings	Finding[]	Yes	Section 6.5; empty for a clean listing
comparison	Comparison	Yes	Section 6.6
scoring	Scoring	Yes	Section 6.7
suggestion	Suggestion|null	Yes	Section 6.8
suggestion_unavailable_reason	string|null	Yes	`NO_MISMATCH_DETECTED`, `UNCERTAIN_OUTPUT`, `UNSUPPORTED_CATEGORY` or `ANALYSIS_INCOMPLETE`

6.11 AnalysisSummaryItem

Field	Type	Required	Description / database source
analysis_run_id	integer	Yes	Run identifier
status	string	Yes	Run status
queue_state	string|null	Yes	Queue assignment
inconsistency_level	string|null	Yes	Level
quality_score	number|null	Yes	Score if computed

Field	Type	Required	Description / database source
finding_count	integer	Yes	Number of findings
has_suggestion	boolean	Yes	True if a `suggested_correction` exists
created_at	string	Yes	Run creation time
completed_at	string|null	Yes	Run end time

6.12 ReviewQueueItem

Field	Type	Required	Description / database source
listing_id	integer	Yes	Listing identifier
analysis_run_id	integer	Yes	Latest completed or partial run
title	string	Yes	Original listing title
seller_category	string	Yes	Raw seller category
image_url	string	Yes	Relative URL of E-10
queue_state	string	Yes	`ready_to_publish`, `needs_review` or `critical_mismatch`
inconsistency_level	string	Yes	`low`, `medium` or `high`
reason_codes	string[]	Yes	Distinct reason codes of the run's findings
suggestion_decision	string|null	Yes	`accept`, `edit`, `ignore`, or null if none recorded or no suggestion
created_at	string	Yes	Run creation time

6.13 CategoryOut

Field	Type	Required	Description / database source
category_code	string	Yes	`supported_category.category_code`; codes pending EDA freeze
display_name	string	Yes	`supported_category.display_name`
is_active	boolean	Yes	`supported_category.is_active`

6.14 ErrorResponse

Every error except the E-01 health response uses this envelope.

Field	Type	Required	Description / database source
error.code	string	Yes	Top-level error code (Section 9.2)
error.message	string	Yes	Plain-language summary suitable for display. A service failure is never worded as a mismatch
error.request_id	string	Yes	Echo of `X-Request-ID` or a generated value
error.details	object[]	Yes	Field-level problems: `{"field": string|null, "issue": string, "message": string}`. Empty when not applicable
error.context	object	Yes	Machine-readable extras such as `listing_id`, `analysis_run_id`, allowed formats. Empty object when none

7. Enumerations

Domain	Allowed values	Source / status
finding.reason_code	`TYPE_MISMATCH`, `COLOR_MISMATCH`, `LOW_CONFIDENCE`, `IMAGE_BLUR`, `MISSING_ATTRIBUTE`, `UNSUPPORTED_CATEGORY`	Database Appendix A; UI/UX 12.3
finding.severity	`Critical`, `Major`, `Minor`, null	UI/UX 12.3; Database dictionary
analysis_run.status	`running`, `completed`, `partial`, `failed`	Database Appendix A
queue_state	`ready_to_publish`, `needs_review`, `critical_mismatch`	Database Appendix A; UI/UX 12.5
inconsistency_level	`low`, `medium`, `high`	Database Appendix A; UI/UX 12.4
attribute	`product_type`, `color`	Database Appendix A
prediction_state	`predicted`, `abstained`	Database dictionary
model_source	`cv`, `clip`, `dominant_color`	**Proposed** codes for the three sources named in the database
source_field	`title`, `description`	Database dictionary
script_route	`en`, `ar`	**Proposed**; FR-008
attribute_scope	`product_type`, `color`, `image`, `listing`	**Proposed** lower-case form of the four scopes in the database
brightness_status	`normal`, `too_dark`, `too_bright`	**Proposed**; not enumerated in the database
decision	`accept`, `edit`, `ignore`	Database Appendix A
text_origin	`seller_input`, `manual_demo`, `synthetic_evaluation`	Database Appendix A
image_origin	Not defined	**Open** (A-04, Database D-09)
comparison row status	`consistent`, `possible_mismatch`, `not_stated`, `uncertain`	UI/UX 13.3
suggestion_unavailable_reason	`NO_MISMATCH_DETECTED`, `UNCERTAIN_OUTPUT`, `UNSUPPORTED_CATEGORY`, `ANALYSIS_INCOMPLETE`	**Proposed**

8. Validation rules

8.1 Where validation happens

The interface performs first-line checks so the seller gets inline messages (UI/UX Section 6.2). The API repeats every check and is the authority. The database protects shape and referential integrity but does not verify any model result (Database Section 12). Validation errors never reach the CV or NLP modules and never create database rows.

8.2 Input rules for E-03

Field	Rule	Issue code	Source / status
image	Present. Missing file does not start analysis.	`REQUIRED_FIELD_MISSING`	FR-001, AC-001; frozen

Field	Rule	Issue code	Source / status
image	Exactly one file part.	MULTIPLE_IMAGES	FR-001; frozen
image	Format is one of the allowed types, determined from the file signature, not the declared content type or file extension. **Proposed** allowed set: JPEG, PNG, WebP.	UNSUPPORTED_IMAGE_FORMAT	FRS leaves formats open (IN-03); configurable ALLOWED_IMAGE_TYPES
image	Size at most MAX_IMAGE_BYTES. **Proposed** starting value 5 MB.	IMAGE_TOO_LARGE	Open; configurable
image	Decodes successfully as an image with non-zero width and height.	IMAGE_UNREADABLE	FR-001; IN-03 open
title	Present and not blank after trimming whitespace.	REQUIRED_FIELD_MISSING / EMPTY_TEXT	FR-002; empty policy open (IN-01)
title	At most MAX_TITLE_CHARS. **Proposed** 200.	FIELD_TOO_LONG	Open; configurable
description	Present and not blank after trimming whitespace.	REQUIRED_FIELD_MISSING / EMPTY_TEXT	FR-002; empty policy open (IN-02)
description	At most MAX_DESCRIPTION_CHARS. **Proposed** 2000.	FIELD_TOO_LONG	Open; configurable
title, description, seller_category	No NUL (\u0000) characters. They cannot be stored in PostgreSQL text.	INVALID_CHARACTERS	Implementation constraint
seller_category	Present, not blank, at most 120 characters (the column length). Not checked against the supported list.	REQUIRED_FIELD_MISSING, FIELD_TOO_LONG	FR-003, FR-015; Database column
text_origin	One of the three allowed values when supplied.	INVALID_VALUE	Database Appendix A
dataset_product_id	At most 80 characters when supplied.	FIELD_TOO_LONG	Database column

8.3 Rules and order of checking

1. Reject a body above the hard request limit with 413 before reading it fully.
2. Reject a body that is not parseable multipart with 400.
3. Check presence of all required parts, then blank text, length, characters and enumeration values. Collect every problem.
4. If an image is present, check count, signature, size and decodability. Collect every problem.
5. If any problem was collected, return one 422 VALIDATION_ERROR listing all of them. The interface focuses the first invalid field and keeps entered data (UI/UX 6.2).
6. Otherwise proceed to storage and analysis.

Text handling. Title and description are stored exactly as submitted and passed to the NLP module unchanged (FR-002, FR-007). Trimming is used only to decide whether a field is blank. Arabic and English text are both accepted as UTF-8; the script route is chosen by the NLP module, not by the client (FR-008). Unknown vocabulary yields no claim rather than an error.

8.4 Rules for other endpoints

Endpoint	Rule
E-04, E-05, E-06, E-10	listing_id is a positive integer. Unknown id gives 404.

Endpoint	Rule
E-06, E-09	`limit` between 1 and 100 and `offset` at least 0, otherwise `VALIDATION_ERROR`.
E-07	`analysis_run_id` is a positive integer. Unknown id gives 404.
E-08	`suggestion_id` is a positive integer. `decision` is one of three values. `edit` needs both final fields, not blank. `accept` and `ignore` must not carry final fields. A second decision gives 409.
E-09	`queue_state`, when given, is one of the three queue values.

9. Error handling

9.1 Principles

- One envelope for all errors (Section 6.14), so the interface needs one handler.
- A service failure is never worded or coded as a mismatch. `ANALYSIS_FAILED` and its message say that the operation failed and that no result was produced (UI/UX 13.4).
- Uncertain output is **not** an error. It is a successful 201 with `uncertain` rows, a `LOW_CONFIDENCE` finding and no suggestion.
- Errors never expose file paths, SQL, stack traces or internal module names. Details go to server logs, correlated by `request_id`.
- Messages are plain language and may be shown directly. Machine decisions use `code` and `issue`, not `message`.

9.2 Top-level error codes

Code	HTTP	Trigger	Interface response
`VALIDATION_ERROR`	422	Any field, image, query or decision-payload rule fails	Inline messages from `details`; focus the first invalid field; keep inputs
`MALFORMED_REQUEST`	400	Body cannot be parsed	Generic request error; allow retry
`REQUEST_TOO_LARGE`	413	Body above the hard limit	Ask for a smaller image
`LISTING_NOT_FOUND`	404	Unknown `listing_id`	Return to the input screen
`ANALYSIS_NOT_FOUND`	404	Unknown `analysis_run_id`	Return to the input screen
`SUGGESTION_NOT_FOUND`	404	Unknown `suggestion_id`	Hide the suggestion card
`IMAGE_NOT_FOUND`	404	File missing from storage	Show a placeholder; offer to resubmit
`DECISION_ALREADY_RECORDED`	409	A decision exists for the suggestion	Show the recorded decision from `context`
`ANALYSIS_FAILED`	503	No module produced a usable result	Failure screen; fields preserved; Retry uses E-05 with `context.listing_id`
`ANALYSIS_TIMEOUT`	504	Pipeline exceeded the time limit	Same as `ANALYSIS_FAILED`
`SERVICE_UNAVAILABLE`	503	Database or storage unreachable	Failure screen; entered fields preserved in the interface
`INTERNAL_ERROR`	500	Unexpected server error	Failure screen; quote `request_id`

9.3 Field-level issue codes

These appear in `error.details[].issue` under `VALIDATION_ERROR`.

Issue code	Meaning	Suggested message
REQUIRED_FIELD_MISSING	A required part or parameter is absent	Matches the UI copy, for example "Enter a product title."
EMPTY_TEXT	Present but blank after trimming (policy pending)	"Enter a product description."
FIELD_TOO_LONG	Exceeds the maximum length	"Shorten this field to N characters or fewer."
INVALID_CHARACTERS	Contains characters that cannot be stored	"Remove unsupported characters."
INVALID_VALUE	Not an allowed enumeration value	"Choose one of the allowed values."
MULTIPLE_IMAGES	More than one image part	"Upload one image only."
UNSUPPORTED_IMAGE_FORMAT	Signature is not an allowed format	"This file type is not supported. Upload a JPEG, PNG or WebP image."
IMAGE_TOO_LARGE	Above MAX_IMAGE_BYTES	"This image is too large. Upload a smaller file."
IMAGE_UNREADABLE	Cannot be decoded	"This image cannot be analysed. Replace the image."
DECISION_FIELDS_REQUIRED	Edit without both final fields	"Enter the final title and description."
DECISION_FIELDS_NOT_ALLOWED	Accept or Ignore with final text	"Final text is only used when editing."

10. Worked examples

The examples follow the demonstration scenarios in the UI/UX package (black sneaker image, text claiming white). Identifiers, confidence values, versions and category codes are illustrative placeholders, not model output.

10.1 Submit a listing (mismatch found)

Request.

```
POST /api/v1/listings/analyze HTTP/1.1
Content-Type: multipart/form-data; boundary=----sl
X-Request-ID: 7d2f0c1e

image              = (file) sample-black-sneaker.jpg   [image/jpeg]
title              = Men's white running sneakers
description        = Lightweight white sneakers with breathable mesh
                     and a cushioned sole, ideal for daily running.
seller_category    = Sports Shoes
```

Equivalent command line:

```
curl -X POST http://localhost:8000/api/v1/listings/analyze \
  -H "X-Request-ID: 7d2f0c1e" \
  -F "image=@sample-black-sneaker.jpg;type=image/jpeg" \
  -F "title=Men's white running sneakers" \
  -F "description=Lightweight white sneakers with breathable mesh and a cushioned sole, ideal for daily
running." \
  -F "seller_category=Sports Shoes"
```

Response, 201 Created. Header `Location: /api/v1/analyses/101`.

```
{
  "analysis_run_id": 101,
  "listing_id": 57,
  "status": "completed",
  "created_at": "2026-10-30T09:14:02Z",
  "completed_at": "2026-10-30T09:14:05Z",
  "listing": {
    "listing_id": 57,
    "title": "Men's white running sneakers",
```

```
    "description": "Lightweight white sneakers with breathable mesh and a cushioned sole, ideal for daily
running.",
    "seller_category": "Sports Shoes",
    "text_origin": "seller_input",
    "image_origin": "seller_upload",
    "dataset_product_id": null,
    "image_url": "/api/v1/listings/57/image",
    "image_sha256": "9f2c...e41a",
    "created_at": "2026-10-30T09:14:02Z"
  },
  "versions": {
    "cv_model_version": "cv-demo-0.1",
    "nlp_lexicon_version": "lex-demo-0.1",
    "ruleset_version": "rules-demo-0.1"
  },
  "image_quality": {
    "width_px": 60, "height_px": 80,
    "is_blurred": false, "brightness_status": "normal"
  },
  "visual_evidence": {
    "product_type": {
      "prediction_id": 501, "attribute": "product_type", "model_source": "cv",
      "prediction_state": "predicted", "value": "Sports Shoes",
      "category_code": "sports_shoes", "confidence": 0.94, "is_selected": true
    },
    "color": {
      "prediction_id": 502, "attribute": "color", "model_source": "cv",
      "prediction_state": "predicted", "value": "black",
      "category_code": null, "confidence": 0.94, "is_selected": true
    },
    "other_predictions": []
  },
  "text_claims": [
    {"claim_id": 301, "attribute": "product_type", "source_field": "title",
     "matched_text": "running sneakers", "canonical_value": "Sports Shoes", "script_route": "en"},
    {"claim_id": 302, "attribute": "color", "source_field": "title",
     "matched_text": "white", "canonical_value": "white", "script_route": "en"}
  ],
  "findings": [
    {"finding_id": 701, "reason_code": "COLOR_MISMATCH", "attribute_scope": "color",
     "severity": "Major", "confidence": 0.94,
     "visual_value": "black", "claimed_value": "white",
     "explanation": "The image shows black but the listing text says white. These are different color
families."}
  ],
  "comparison": {
    "rows": [
      {"attribute": "product_type",
       "image_supported": {"available": true, "value": "Sports Shoes", "confidence": 0.94},
       "text_claim": {"stated": true, "value": "Sports Shoes"},
       "status": "consistent", "finding_id": null},
      {"attribute": "color",
       "image_supported": {"available": true, "value": "black", "confidence": 0.94},
       "text_claim": {"stated": true, "value": "white"},
       "status": "possible_mismatch", "finding_id": 701}
    ],
    "summary": {"attributes_checked": 2, "consistent": 1, "needs_review": 1, "uncertain": 0}
  },
  "scoring": {"quality_score": null, "inconsistency_level": "medium", "queue_state": "needs_review"},
  "suggestion": {
    "suggestion_id": 88,
    "original": {
      "title": "Men's white running sneakers",
      "description": "Lightweight white sneakers with breathable mesh and a cushioned sole, ideal for daily
running."
    },
    "suggested": {
      "title": "Men's black running sneakers",
      "description": "Lightweight black sneakers with breathable mesh and a cushioned sole, ideal for daily
running."
    },
    "changed_terms": [
      {"field": "title", "from": "white", "to": "black"},
      {"field": "description", "from": "white", "to": "black"}
    ],
    "change_explanation": "The image supports the color black, but the listing text says white. Only the
color term was changed.",
    "decision": null
  },
  "suggestion_unavailable_reason": null
```

```
}
```

How the interface uses it. The banner reads `scoring.queue_state` (Needs review) and the chips read `findings[0].reason_code` (COLOR_MISMATCH), `findings[0].severity` (Major), `scoring.inconsistency_level` (Medium) and the queue. The tiles read `comparison.summary`. The highlighted words come from `suggestion.changed_terms`. This matches UI/UX Figure 6.

10.2 Validation errors (empty form)

Response, 422 Unprocessable Entity. Messages reuse the copy in UI/UX Figure 3.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "4 fields need attention.",
    "request_id": "7d2f0c1e",
    "details": [
      {"field": "image", "issue": "REQUIRED_FIELD_MISSING", "message": "Upload a product image."},
      {"field": "title", "issue": "REQUIRED_FIELD_MISSING", "message": "Enter a product title."},
      {"field": "description", "issue": "REQUIRED_FIELD_MISSING", "message": "Enter a product
description."},
      {"field": "seller_category", "issue": "REQUIRED_FIELD_MISSING", "message": "Select the seller-stated
category."}
    ],
    "context": {}
  }
}
```

10.3 Unsupported image format

A file whose signature is not JPEG, PNG or WebP (for example a PDF renamed to `.jpg`). **Response, 422.**

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The image cannot be used.",
    "request_id": "a41b77c0",
    "details": [
      {"field": "image", "issue": "UNSUPPORTED_IMAGE_FORMAT",
       "message": "This file type is not supported. Upload a JPEG, PNG or WebP image."}
    ],
    "context": {"allowed_formats": ["image/jpeg", "image/png", "image/webp"]}
  }
}
```

10.4 Failed analysis and retry

Both branches failed. The listing and a `failed` run were stored. **Response, 503 Service Unavailable.**

```
{
  "error": {
    "code": "ANALYSIS_FAILED",
    "message": "Analysis could not be completed. This is a service failure, not a mismatch finding. No
result was produced.",
    "request_id": "c93d10aa",
    "details": [],
    "context": {
      "listing_id": 58,
      "analysis_run_id": 102,
      "failed_stages": ["cv", "nlp"],
      "retry": {"method": "POST", "path": "/api/v1/listings/58/analyses"}
    }
  }
}
```

The interface keeps the entered fields and offers Retry, which calls E-05 with `listing_id` 58. No image is re-uploaded. A successful retry creates run 103; run 102 remains as history.

10.5 Uncertain output (excerpt)

The color prediction abstained. No match or mismatch is inferred and no suggestion is offered. This matches UI/UX Figure 8 (reason code LOW_CONFIDENCE, severity not assigned, inconsistency Low, queue Needs review). Only the changed parts are shown.


```
{
  "status": "completed",
  "visual_evidence": {
    "product_type": { "...": "predicted, Sports Shoes, confidence 0.91" },
    "color": {
      "prediction_id": 512, "attribute": "color", "model_source": "cv",
      "prediction_state": "abstained", "value": null,
      "category_code": null, "confidence": 0.41, "is_selected": true
    },
    "other_predictions": []
  },
  "findings": [
    {"finding_id": 712, "reason_code": "LOW_CONFIDENCE", "attribute_scope": "color",
     "severity": null, "confidence": 0.41, "visual_value": null, "claimed_value": "white",
     "explanation": "The color prediction is below the confidence threshold, so no match or mismatch is
reported."}
  ],
  "comparison": {
    "rows": [
      {"attribute": "product_type", "status": "consistent", "...": "..."},
      {"attribute": "color",
       "image_supported": {"available": false, "value": null, "confidence": 0.41},
       "text_claim": {"stated": true, "value": "white"},
       "status": "uncertain", "finding_id": 712}
    ],
    "summary": {"attributes_checked": 2, "consistent": 1, "needs_review": 0, "uncertain": 1}
  },
  "scoring": {"quality_score": null, "inconsistency_level": "low", "queue_state": "needs_review"},
  "suggestion": null,
  "suggestion_unavailable_reason": "UNCERTAIN_OUTPUT"
}
```


10.6 Record an Edit decision, then a duplicate

Request. The seller edited the suggested description before approving it.


```
POST /api/v1/suggestions/88/decision HTTP/1.1
Content-Type: application/json


{
  "decision": "edit",
  "final_title": "Men's black running sneakers",
  "final_description": "Lightweight black sneakers with breathable mesh and a cushioned sole, ideal for
everyday running."
}
```


Response, 201 Created.


```
{
  "decision_id": 41,
  "suggestion_id": 88,
  "analysis_run_id": 101,
  "listing_id": 57,
  "decision": "edit",
  "final_title": "Men's black running sneakers",
  "final_description": "Lightweight black sneakers with breathable mesh and a cushioned sole, ideal for
everyday running.",
  "decided_at": "2026-10-30T09:20:11Z",
  "resulting_text": {
    "title": "Men's black running sneakers",
    "description": "Lightweight black sneakers with breathable mesh and a cushioned sole, ideal for
everyday running.",
    "basis": "seller_edit"
  },
  "published": false
}
```


A second request for suggestion 88 returns 409:


```
{
  "error": {
```

```
    "code": "DECISION_ALREADY_RECORDED",
    "message": "A decision has already been recorded for this suggestion.",
    "request_id": "e5502b19",
    "details": [],
    "context": {"decision_id": 41, "decision": "edit"}
  }
}
```

An invalid Accept that carries final text returns 422 with issue `DECISION_FIELDS_NOT_ALLOWED` on `final_title`.

11. Coordination with Database, Architecture and UI/UX

11.1 Database (Member 2): field and rule alignment

API field names for stored data reuse the database column names. Where the API adds a derived field, it is marked as derived and has no column.

API element	Database element	Alignment note
`ListingOut`	`product_listing`	`image_ref` replaced by `image_url`; text stored unchanged
`AnalysisResult.status`	`analysis_run.status`	Same four values. UI/UX contract `analysis_status: "ok"` maps to `completed`
`scoring.*`	`analysis_run.quality_score`, `inconsistency_level`, `queue_state`	Lower-case snake values taken from Database Appendix A
`image_quality.*`	`analysis_run.image_width_px`, `image_height_px`, `is_blurred`, `brightness_status`	Null means not obtained
`visual_evidence`	`visual_prediction`	Selected rows are promoted to `product_type` and `color`; unique selected row per run and attribute
`text_claims[]`	`text_claim`	Direct mapping
`findings[]`	`analysis_finding`	Severity null for review flags, as in the database
`suggestion`	`suggested_correction`	At most one per run; `change_explanation` is NOT NULL in the database
`DecisionOut`	`seller_decision`	One per suggestion; final fields only for `edit`
`CategoryOut`	`supported_category`	Direct mapping
`comparison`, `changed_terms`, `resulting_text`	None	Derived at response time; not stored

Database operations per endpoint

Endpoint	Reads	Writes
E-01	Probe query	None
E-02	`supported_category`	None
E-03	`supported_category` (unsupported-category check)	`product_listing`, `analysis_run`, `visual_prediction`, `text_claim`, `analysis_finding`, `suggested_correction`
E-04, E-10	`product_listing`, `analysis_run`	None

Endpoint	Reads	Writes
E-05	`product_listing`, `supported_category`	New `analysis_run` and children
E-06, E-07	`analysis_run` and children, `seller_decision`	None
E-08	`suggested_correction`, `seller_decision`	`seller_decision`
E-09	Latest-run query over `analysis_run`, with joins	None

Transaction rule. Evidence rows and the run update for one analysis are written in a single transaction (step 8 of E-03). The listing and the `running` run are committed earlier so that a failure can be recorded and retried. Only the FastAPI backend writes to the database; the Streamlit interface never connects to it (Database Section 16).

11.2 Deletion behaviour

The API exposes no deletion endpoint, which is consistent with the database's restrictive foreign keys: a listing with runs cannot be deleted, and a suggestion with a decision cannot be deleted. If a retention policy is approved later, deletion must be added as a separate authorised administrative path, not as part of this contract.

11.3 Architecture (Member 3): module and object alignment

The database specification names the typed objects passed between modules and states that the names and endpoint definitions are Member 4 decisions. This table proposes the mapping.

Architecture object	API schema	Produced by
ListingInput	E-03 multipart parts; `ListingOut`	FastAPI validation
VisualEvidence	`visual_evidence`, `image_quality`	CV module (type, color, image quality)
TextClaims	`text_claims`	NLP module (English rules, fixed Arabic lexicon)
MismatchReport	`findings`, `comparison`	Mismatch Detection
ListingAssessment	`scoring`	Scoring and Triage
SuggestedFix	`suggestion`	Suggestion module (templates, no LLM)
SellerDecision	`DecisionOut`	E-08 handler

The CV and NLP branches run in parallel and in memory; their intermediate computations are not persisted (Database Figure 3). Model weights, thresholds, lexicons and templates live in the external Model and Lexicon Store. The API records only identifiers in `versions`.

11.4 UI/UX (Member 5): contract mapping

UI/UX Section 13.1 proposes a response shape. The API keeps its content but is aligned to the database; this table lets the interface switch from the hard-coded scenarios to the API without changing screens.

UI/UX contract element	API field	Note
`analysis_status: "ok"`	`status = completed`	`partial` and `failed` are new states the interface must handle
`cv.type`, `cv.color`	`visual_evidence.product_type.value`, `.color.value`	Null `value` means show "Unavailable"

UI/UX contract element	API field	Note
`cv.confidence.type`, `.color`	`...confidence` on each prediction	Moved next to each value
`cv.quality.*`	`image_quality.*`	`resolution_ok` and `brightness_ok` are not stored; the interface reads width, height and `brightness_status`
`nlp.claimed_type`, `.claimed_color`	`comparison.rows[].text_claim.value`	"Not stated" when `stated` is false (FR-012)
`nlp.language`	`text_claims[].script_route`	Per claim; absent if no claim was extracted
`comparison.mismatches[]`	`findings[]` with a mismatch reason code	Adds `finding_id`, `confidence`, `attribute_scope`
`comparison.missing_attributes[]`	`findings[]` with `MISSING_ATTRIBUTE`; rows with `not_stated`	
`scoring.queue`	`scoring.queue_state`	Values `ready_to_publish`, `needs_review`, `critical_mismatch`; the interface supplies display labels
`scoring.inconsistency_level`	`scoring.inconsistency_level`	Lower-case; the interface capitalises for display
`suggestion.title`, `.description`	`suggestion.suggested.title`, `.description`	Adds `original`, `changed_terms`, `change_explanation`
Count tiles	`comparison.summary`	Never typed by hand
Review chips	`findings[0]`, `scoring`	One reason code per surfaced warning
Category dropdown	E-02	Replaces the provisional list once the five classes are seeded
Accept, Edit, Ignore	E-08	No publish call
Cancel demo	None	The synchronous API has no cancellation; see A-01
nlp.claimed_type, nlp.claimed_color	comparison.rows[].text_claim.value (traceable to text_claims[] by claim_id)	Use the comparison row as the primary display value; text_claims[] is the evidence collection. A missing claim is displayed as 'Not stated' only when stated=false.
comparison.rows[]	comparison.rows[]	The API comparison object is the

UI/UX contract element	API field	Note
		normalized UI-ready comparison. Render its status and values; do not independently recalculate match status in Streamlit.
analysis progress percentage/stages	No field in the synchronous API contract	Use indeterminate progress and generic truthful status text while the request is pending. Do not fabricate percentages or stage completion. No production cancel action is supported.
empty seller-category options	E-02 response: items=[] and count=0	Show a configuration-unavailable state and block normal submission; do not use provisional candidate classes as approved options.

11.5 Discrepancies found and proposed resolutions

#	Observation	Proposed resolution	Owner
1	UI/UX contract uses `analysis_status: ok`; database uses `running/completed/partial/failed`.	API exposes the database values. Interface maps `completed` to its "ok" path.	Member 5
2	UI/UX shows Low/Medium/High and "Needs review"; database stores lower-case codes.	API returns database codes. Display labels belong to the interface.	Member 5
3	UI/UX suggestion has only title and description; database requires `change_explanation` and an id.	API adds `suggestion_id`, `change_explanation`, `original` and derived `changed_terms`.	Member 5, Member 2
4	The uncertain-output prototype shows a suggestion card with disabled buttons; UI/UX 13.3 says to hide the card when nothing supported is wrong.	API returns `suggestion: null` with a reason code. The interface may render a disabled placeholder from the reason.	Member 5
5	UI/UX shows real-looking stage progress; the synchronous API cannot report real stages, and UI/UX requires only truthful stages.	Interface shows a generic busy state, or the team adds a polling field (A-01).	Member 3, Member 5
6	The database enumerates `queue_state` and `status` but not `attribute_scope`, `brightness_status`, `model_source`, `script_route` or `image_origin`.	API proposes codes (Section 7). Member 2 should add matching CHECK constraints or confirm free text.	Member 2
7	The seller category is stored raw; the interface sends a dropdown value.	Decide whether the interface sends the display name or the code. API accepts free text up to 120 characters either way (A-05).	Member 2, Member 5
8	Database empty-text rules IN-01 and IN-02 are unenforced; UI/UX prototype rejects blank text.	API rejects blank title and description until the team decides otherwise (A-03).	Requirements owner

#	Observation	Proposed resolution	Owner
9	Database item D-09: seller upload versus dataset-only image provenance is unresolved.	API accepts uploads and sets `image_origin`; value list pending (A-04).	FRS owner

Before implementation is considered ready, compare every Appendix A Pydantic field and validator against Sections 6–9, database column names/check constraints, and JSON examples. Use independent default factories for mutable list/dict fields, ensure E-03 multipart parameters match the documented field names, and convert FastAPI validation exceptions to the documented ErrorResponse envelope.

I. Implementation contract checks

E-06 and E-09 use limit (1–100, default 20) and offset (0 or greater, default 0). The UI should preserve these values when requesting subsequent pages. E-08 accepts one immutable decision per suggestion; a second decision returns 409 DECISION_ALREADY_RECORDED. Accept and Ignore must not include final text; Edit must include both non-blank final_title and final_description. No decision publishes or updates a marketplace listing.

H. Pagination and decision behaviour

E-10 returns image bytes, not a JSON envelope, with the stored Content-Type and X-Content-Type-Options: nosniff. The UI should use image_url from ListingOut rather than construct a storage path. An unknown listing_id returns the standard 404 error envelope. If the listing exists but the referenced image file is missing, return a not-found response without exposing image_ref or a filesystem path.

G. Binary image endpoint

Let the HTTP client set the multipart Content-Type boundary; do not set it manually. On success, use the returned listing_id and analysis_run_id and follow the Location header to E-07. On validation failure, retain seller-entered values and display each error detail against its field.

```
curl -X POST "http://localhost:8000/api/v1/listings/analyze" \
  -F "image=@./sample.jpg" \
  -F "title=Black leather handbag" \
  -F "description=Structured black handbag with top handles" \
  -F "seller_category=Handbags"
```

The E-03 multipart field names are image, title, description and seller_category. Optional fields are text_origin, image_origin and dataset_product_id. Example (replace the local file path with a real image):

F. End-to-end multipart request example

The proposed JPEG/PNG/WebP allow-list, 5 MB image limit, 200-character title limit and 2,000-character description limit are not fixed by the supplied source specifications. Keep them configurable and treat them as proposals until approved. Once agreed, the UI and API must use the same values; the API remains authoritative and returns the documented validation envelope.

E. Configurable validation limits

Use AnalysisResult.comparison.rows as the primary UI-ready comparison; use text_claims as the evidence collection. Read image-supported type and color from visual_evidence.product_type.value and visual_evidence.color.value, with null rendered as 'Unavailable'. Read the queue code from scoring.queue_state and map it to the UI label; do not recompute mismatch, counts or queue state in Streamlit. Read findings[] for reason_code, severity and attribute_scope, and comparison.summary for count tiles. Keep API/database enum codes unchanged in transport and apply display labels only in the UI.

D. UI field mapping and rendering

For each listing, first select its newest completed or partial analysis run; exclude failed and running runs from this candidate set. Apply the requested queue_state filter to that selected run. A newer failed retry therefore does not hide an earlier usable result. This aligns the API with the Database Design Specification's latest relevant completed/partial run rule and the UI/UX Review Queue workflow. Severity/tie-breaking and combined-flag precedence remain with the Scoring and Triage module and must not be invented by the API layer.

C. Review Queue selection rule

E-02 is the sole source of supported categories. Before the five EDA-approved classes are seeded, the response is {"items": [], "count": 0}. The normal seller flow should explain that category options are not configured and prevent submission rather than silently use the illustrative candidates in the UI/UX proposal. The seller_category API field remains a raw required string for controlled evaluation/demo use, subject to explicit team agreement.

B. Category list is empty until EDA freeze

E-03 and E-05 remain synchronous in this MVP. The UI/UX package's four-stage pipeline and percentage bar are prototype presentation, not data the current API can truthfully provide. While awaiting the HTTP response, show

an indeterminate progress indicator with concise status text such as 'Analysis in progress', disable duplicate submission, and retain submitted form data if the request fails. Do not display a numeric percentage, claim a stage is complete, or offer production cancellation unless an asynchronous job/status contract is approved and implemented.

A. Synchronous analysis and progress UI

These clarifications make the existing endpoint contract implementable without changing the endpoint set or the database's immutable listing/run model.

11.6 Integration clarifications for Member 5 and Member 3

12. Requirements traceability

Coverage is structural: the API provides the input, output or control that the requirement needs. Inference, rule logic and scoring are implemented in backend modules, not in the API layer. Requirement statements are those quoted in the Database Design Specification.

Requirement	Endpoint and field	Coverage
FR-001 one product image	E-03 `image`; E-10	Proposed, mapped
FR-002 title and description	E-03 `title`, `description`	Proposed, mapped
FR-003 seller category	E-03 `seller_category`; E-02	Proposed, mapped
FR-004 product type prediction	`visual_evidence.product_type`	Proposed, mapped
FR-005 color prediction	`visual_evidence.color`	Proposed, mapped
FR-006 image quality checks	`image_quality`; `IMAGE_BLUR` finding	Proposed, mapped
FR-007 claim extraction	`text_claims`	Proposed, mapped
FR-008 Arabic or English route	`text_claims[].script_route`	Proposed, mapped
FR-009 image-text comparison	`comparison`, `findings`	Proposed, mapped
FR-010 type mismatch	`findings` with `TYPE_MISMATCH`	Proposed, mapped
FR-011 color mismatch	`findings` with `COLOR_MISMATCH`	Proposed, mapped
FR-012 missing attribute	`not_stated` rows; `MISSING_ATTRIBUTE`	Proposed, optional
FR-013 confidence	`confidence` on predictions and findings	Proposed, mapped
FR-014 low confidence	`LOW_CONFIDENCE`; `uncertain` rows; queue	Proposed, mapped
FR-015 unsupported category	`UNSUPPORTED_CATEGORY`; `suggestion_unavailable_reason`	Proposed, mapped
FR-016 reason codes and explanation	`findings[].reason_code`, `explanation`	Proposed, mapped
FR-017 severity	`findings[].severity`	Proposed, mapped
FR-018 quality score	`scoring.quality_score`	Proposed, optional
FR-019 inconsistency level	`scoring.inconsistency_level`	Proposed, mapped
FR-020 review queue	`scoring.queue_state`; E-09	Proposed, mapped
FR-021 template suggestion	`suggestion`	Proposed, mapped
FR-022 before and after	`suggestion.original`, `suggested`, `changed_terms`, `change_explanation`	Proposed, mapped
FR-023 accept	E-08 `accept`	Proposed, mapped
FR-024 edit	E-08 `edit`	Proposed, mapped
FR-025 ignore	E-08 `ignore`	Proposed, mapped

Requirement	Endpoint and field	Coverage
UC-13 review queue	E-09	Proposed; queue rules open

NFR identifiers could not be verified because the NFR document was not supplied. Error handling, input validation and the no-publish rule are intended to support the reliability, usability and security requirements; confirm identifiers before submission.

13. Open decisions and assumptions

ID	Open item	Provisional treatment in this specification	Owner
A-01	Synchronous request versus asynchronous job with progress (production behaviour clarified)	MVP remains synchronous. Do not show a fabricated percentage, stage completion, or cancellation. While the request is pending, show an indeterminate progress indicator and truthful generic status text; disable duplicate submission. The four named stages and percentage in the UI/UX package are prototype presentation only until real progress events are implemented. Production cancellation is not available in this contract. A future asynchronous design requires a job resource, status/progress endpoint and persistence/cache decision.	Members 3, 5
A-02	Hard request limit and analysis timeout	Configurable constants; no numeric value fixed.	Member 3
A-03	Empty title or description (IN-01, IN-02)	Rejected with a validation error, as in the UI/UX prototype.	Requirements owner
A-04	Image origin values and the seller-upload versus dataset-only question (D-09)	Optional `image_origin` with server default; allowed values not fixed.	FRS owner, Member 2
A-05	Seller category value sent by the interface and behaviour while `supported_category` is empty	Seller category remains a required raw string (maximum 120 characters) and is not rejected merely because supported_category is empty. E-02 returns an empty list. The normal seller UI must explain that category options are not configured and block normal submission until the five EDA-approved classes are seeded; do not silently substitute illustrative candidates. Evaluation-only override requires explicit team agreement.	Members 2, 3, 5
A-06	Queue precedence and latest-run selection	Resolved for this API version: select the latest completed or partial run per listing, excluding failed and running runs from the candidate set; then filter by queue_state. A newer failed/running retry does not hide the latest usable result. Ties and combined-flag precedence remain owned by the Scoring and Triage module.	Member 3 (implementation); Member 5 (UI integration)
A-07	Decision edit history and retention (D-05)	One immutable decision per suggestion; no delete or update endpoint.	Product team

ID	Open item	Provisional treatment in this specification	Owner
A-08	Enumeration codes for `attribute_scope`, `brightness_status`, `model_source`, `script_route`	Proposed values in Section 7.	Members 2, CV, NLP
A-09	Image formats, size limit and text length limits (IN-03)	JPEG, PNG and WebP; 5 MB; title 200 characters; description 2,000 characters remain configurable proposals, not frozen source requirements. Keep client and server values in one agreed configuration and update both together after Member 3/requirements approval.	Member 3
A-10	Authentication and reviewer identity (D-10)	None. Review Queue is open to any local caller.	Member 5
A-11	Database engine (D-01)	PostgreSQL assumed, as in the database specification. The API contract does not depend on it.	Members 2, 3

14. API acceptance test checklist

A starting set for the test owner (Member 6). None of these has been executed.

Test	Endpoint	Scenario	Expected result
API-01	E-03	Valid image and text, mismatching color	201; `COLOR_MISMATCH`, `Major`; queue `needs_review`; suggestion present
API-02	E-03	Valid listing, no mismatch	201; no findings; two `consistent` rows; `suggestion` null with `NO_MISMATCH_DETECTED`
API-03	E-03	Empty form	422 with one detail per required field; nothing stored
API-04	E-03	Blank title (spaces only)	422 `EMPTY_TEXT` for `title`
API-05	E-03	Non-image file renamed `.jpg`	422 `UNSUPPORTED_IMAGE_FORMAT`
API-06	E-03	Corrupt JPEG	422 `IMAGE_UNREADABLE`
API-07	E-03	Two image parts	422 `MULTIPLE_IMAGES`
API-08	E-03	Blurred image	201; `IMAGE_BLUR` finding; queue `needs_review`; not an error
API-09	E-03	Unsupported seller category	201; `UNSUPPORTED_CATEGORY`; no forced type; no suggestion
API-10	E-03	Both modules forced to fail	503 `ANALYSIS_FAILED`; run `failed`; listing kept; error is not coded as a mismatch
API-11	E-03	One module forced to fail	201 with `status: partial`; queue `needs_review`; no suggestion
API-12	E-05	Retry after API-10	201; new run id; earlier run unchanged
API-13	E-08	Accept, Edit and Ignore on separate suggestions	201 each; `published: false`; `final_*` only for Edit
API-14	E-08	Second decision on the same suggestion	409 `DECISION_ALREADY_RECORDED`
API-15	E-08	Edit without final text; Accept with final text	422 `DECISION_FIELDS_REQUIRED`; 422 `DECISION_FIELDS_NOT_ALLOWED`
API-16	E-09	Queue filtered by `needs_review`	Only latest completed or partial run per listing; no failed runs

Test	Endpoint	Scenario	Expected result
API-17	E-02	Before and after seeding the five classes	Empty list; then exactly five active items
API-18	E-07	Unknown id	404 `ANALYSIS_NOT_FOUND` in the standard envelope
API-19	All	Inspect error bodies	No file paths, SQL or stack traces; `request_id` present

Appendix A. Pydantic model sketch

Illustrative FastAPI request and error models for the endpoints with JSON bodies. The multipart form of E-03 is declared with `Form(...)` and `UploadFile` parameters rather than a model. This is a sketch for the implementer; it has not been run.

```
from typing import Literal, Optional
from pydantic import BaseModel, model_validator

class DecisionRequest(BaseModel):
    decision: Literal["accept", "edit", "ignore"]
    final_title: Optional[str] = None
    final_description: Optional[str] = None

    @model_validator(mode="after")
    def check_final_fields(self):
        has_title = bool(self.final_title and self.final_title.strip())
        has_desc = bool(self.final_description and self.final_description.strip())
        if self.decision == "edit":
            if not (has_title and has_desc):
                raise ValueError("DECISION_FIELDS_REQUIRED")
        elif self.final_title is not None or self.final_description is not None:
            raise ValueError("DECISION_FIELDS_NOT_ALLOWED")
        return self

class ErrorDetail(BaseModel):
    field: Optional[str] = None
    issue: str
    message: str

class ErrorBody(BaseModel):
    code: str
    message: str
    request_id: str
    details: list[ErrorDetail] = []
    context: dict = {}

class ErrorResponse(BaseModel):
    error: ErrorBody

class DecisionOut(BaseModel):
    decision_id: int
    suggestion_id: int
    analysis_run_id: int
    listing_id: int
    decision: Literal["accept", "edit", "ignore"]
    final_title: Optional[str]
    final_description: Optional[str]
    decided_at: str
    resulting_text: dict
    published: Literal[False] = False
```

A custom exception handler should convert Pydantic and FastAPI validation errors into the `ErrorResponse` envelope with code `VALIDATION_ERROR`, so that the interface never receives FastAPI's default error format.

End of specification. Version 1.1, proposed for team review.